Task 4: Secure Network Design, Penetration Testing, and Encrypted Traffic Analysis

Prepared by: Kalash Mahajan

Date: 26th January 2026

1. Introduction

This report presents the design of a secure network topology, the execution of a controlled penetration test using the Metasploit framework, and the analysis of encrypted HTTPS traffic using Wireshark and Python. The objective of this task is to demonstrate practical cybersecurity concepts such as network segmentation, vulnerability assessment, and encrypted traffic analysis through hands-on implementation in a lab environment.

1.1 Objectives of the Task

- Design a secure network topology following cybersecurity best practices
- Implement network segmentation using DMZ and VLANs
- Perform vulnerability scanning and exploitation using Metasploit
- Capture and analyze encrypted HTTPS traffic using Wireshark
- Automate traffic metadata analysis using Python

1.2 Tools and Technologies Used

- Kali Linux (Attacker Machine)
- Metasploit Framework
- Metasploitable (Intentionally Vulnerable Target VM)
- Wireshark
- Python (pyshark, matplotlib)
- draw.io (Network Topology Design)

---

2. Secure Network Topology Design

2.1 Overview

The secure network topology is designed for a small organization consisting of approximately 10–20 devices. The organization requires controlled internet access, secure hosting of public-facing services, and internal segmentation to protect critical systems from unauthorized access and lateral movement. The design follows standard cybersecurity best practices such as firewall enforcement, network segmentation, and isolation of untrusted traffic.

2.2 Network Components

The proposed network architecture consists of the following components:

- **Internet**
  Represents external, untrusted networks accessing organizational resources.
- **Perimeter Firewall**
  Acts as the primary security control, filtering inbound and outbound traffic based on defined security rules.
- **DMZ (Demilitarized Zone)**
  A separate network segment used to host public-facing services such as web servers.
- **Public Web Server**
  Hosts externally accessible services while remaining isolated from internal systems.
- **Internal Network**
  Trusted network segment containing organizational users and servers.
- **Employee VLAN**
  Dedicated VLAN for employee workstations with controlled access to internal resources.
- **Guest VLAN**
  Isolated VLAN for guest or visitor devices with internet-only access.
- **Internal Servers**
  Windows and Linux servers providing internal services such as file storage and application hosting.

2.3 Network Topology Diagram

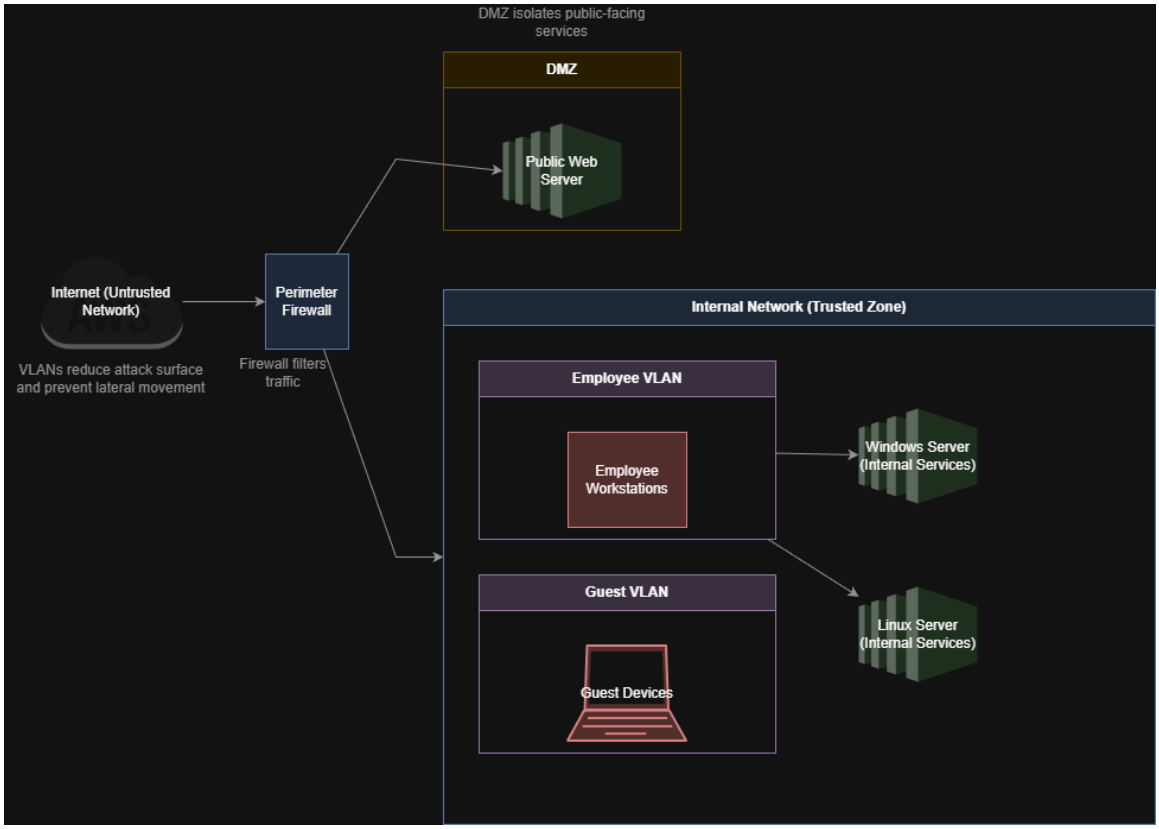

Figure 1: Secure network topology diagram

2.4 Security Design Decisions

- **Firewall Placement**
  - Positioned at the network perimeter to control all inbound and outbound traffic.
  - Prevents direct access from the internet to internal network resources.
- **DMZ Isolation**
  - Public-facing services are isolated from the internal network.
  - Limits the impact of a compromised public server.
  - Prevents attackers from directly accessing internal systems.
- **VLAN Segmentation**
  - Separates employee and guest traffic into distinct network segments.
  - Reduces lateral movement within the network.
  - Minimizes attack surface and improves overall network security.

3. Metasploit Installation and Setup

3.1 Environment Setup

The penetration testing environment was configured using two virtual machines to ensure a controlled and isolated testing setup.

- **Attacker Machine:** Kali Linux

  Used as the penetration testing workstation with Metasploit pre-installed.
- **Target Machine:** Metasploitable

  An intentionally vulnerable virtual machine designed for safe penetration testing and vulnerability assessment.

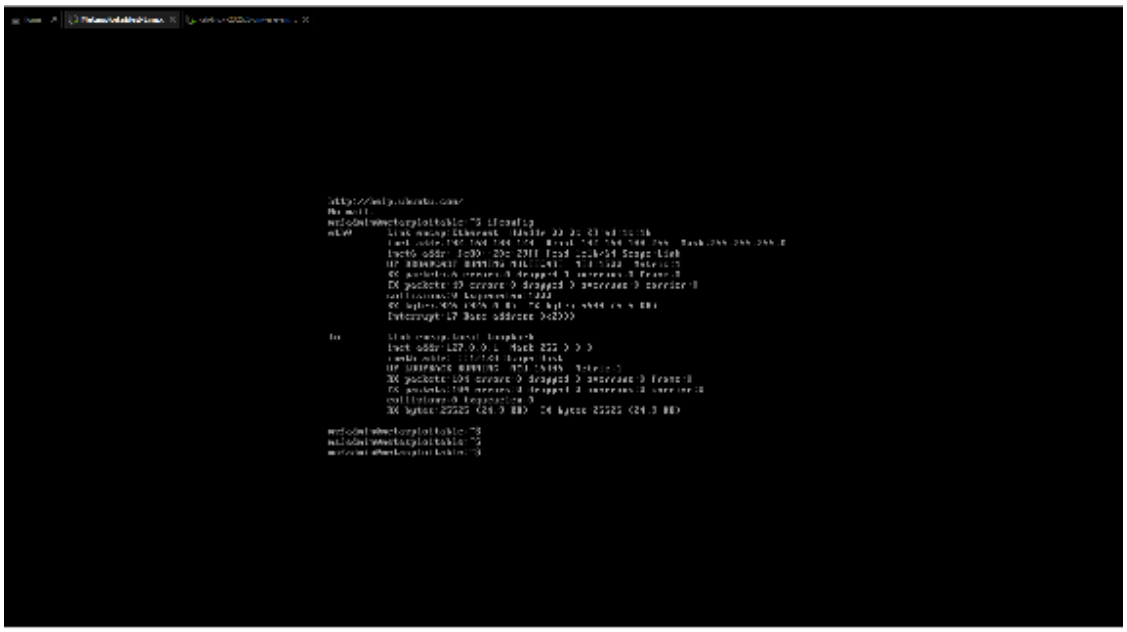

Figure 2: Metasploitable running in the virtual environment

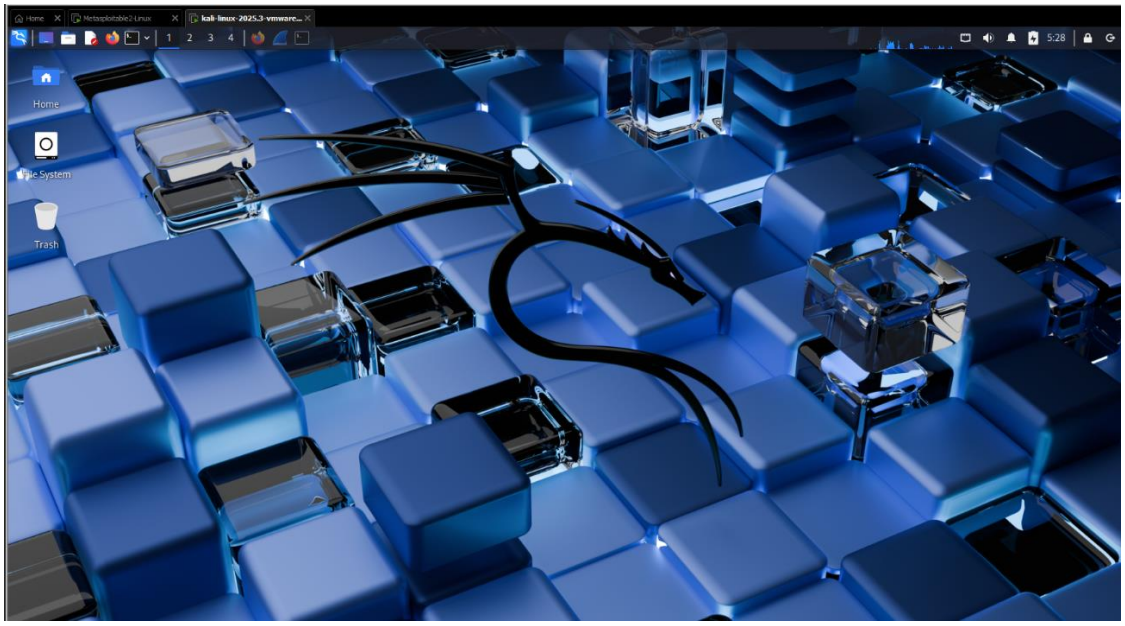

Figure 3: Kali Linux in the virtual environment

3.2 Metasploit Installation

The Metasploit Framework is available by default on Kali Linux. The installation and availability of Metasploit were verified by launching the Metasploit console and confirming the framework version.

- Metasploit was accessed using the msfconsole command
- Successful launch confirmed proper installation
- The framework database was initialized to support scanning and result storage

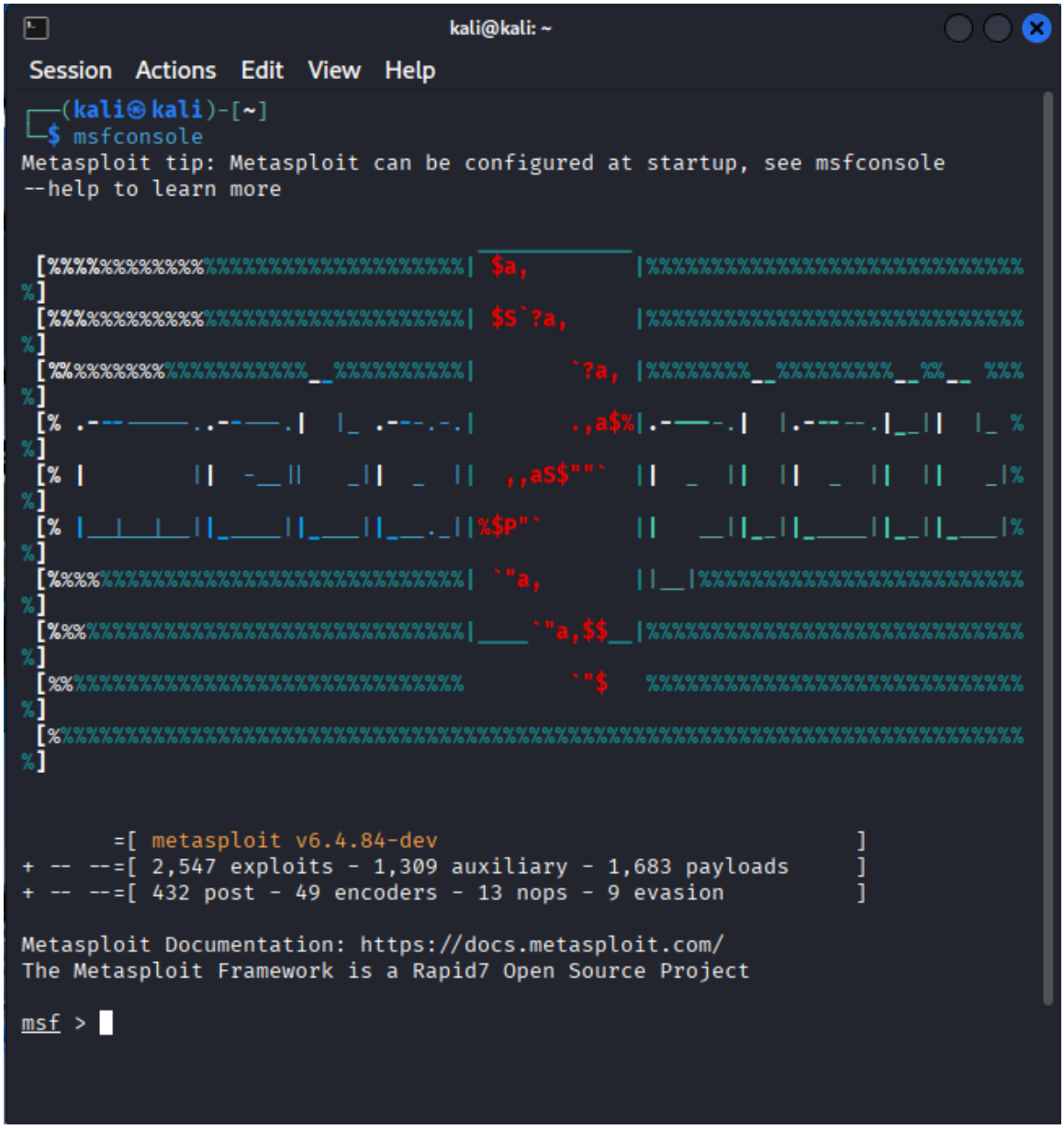

Figure 4: msfconsole showing successful startup and version information

3.3 Understanding Metasploit Basics

Metasploit provides a modular framework for performing penetration testing activities. The following core components were used during this task:

- **Modules**

  Reusable components that perform specific actions such as scanning, exploitation, and post-exploitation.

- **Exploits**

  Code designed to take advantage of known vulnerabilities in target services.

- **Payloads**
  Code executed on the target system after successful exploitation to provide access or perform actions.

These components enable structured and controlled penetration testing within a lab environment.

---

4. Penetration Testing with Metasploit

Vulnerability scanning was performed using the Metasploit Framework to identify open ports and exposed services on the target machine.

- Scanning was conducted from the Kali Linux attacker machine
- Service and version detection was enabled to identify vulnerable software
- Multiple open ports and outdated services were discovered

The scan revealed several services commonly targeted by attackers, including FTP, SSH, HTTP, and database services.

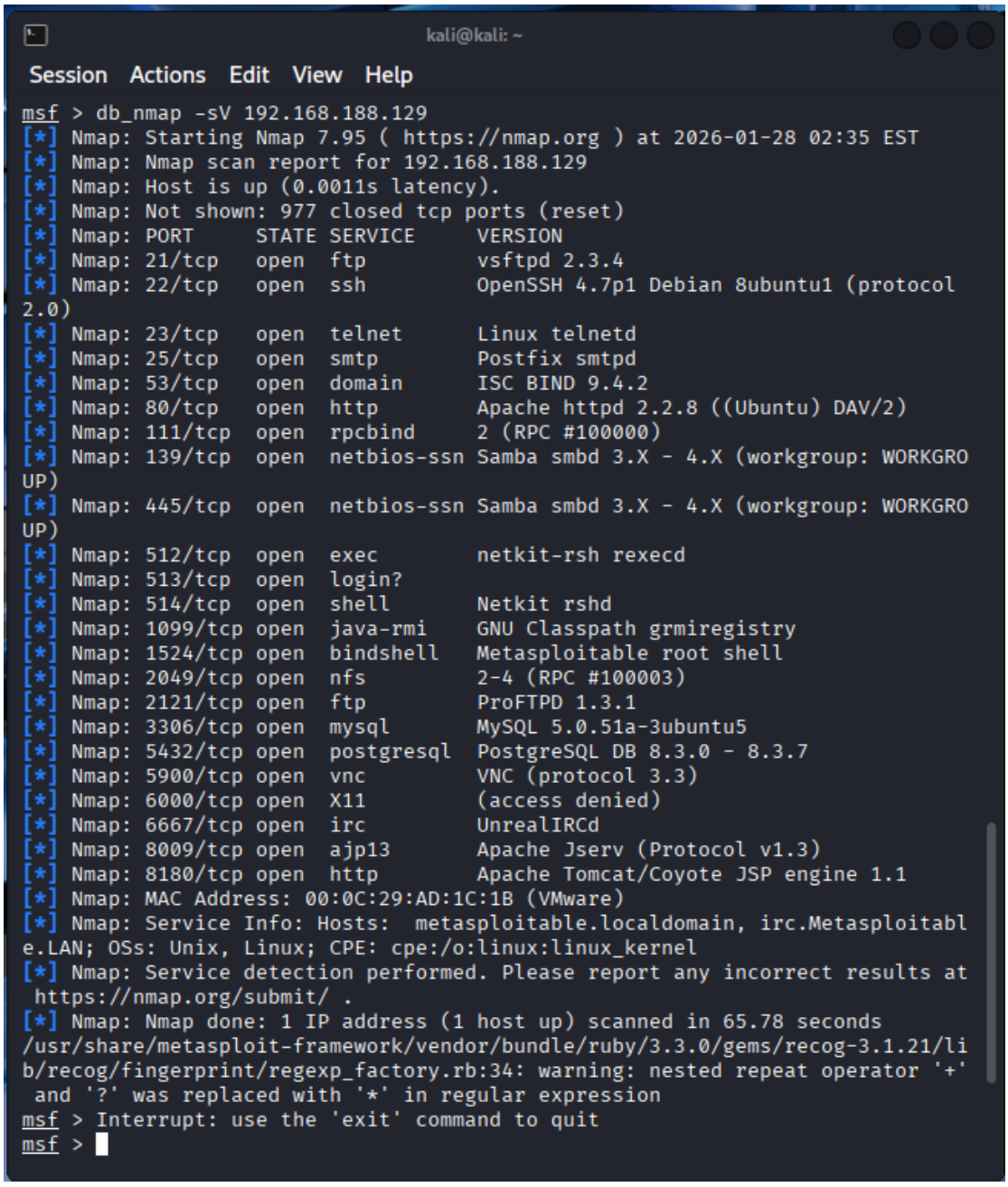

Figure 5: db_nmap -sV scan output from Metasploit

4.3 Exploitation

Based on the scan results, a known vulnerable FTP service was selected for exploitation.

- **Exploit Selected:** vsftpd 2.3.4 backdoor exploit
- **Reason for Selection:**
    - Known and well-documented vulnerability
    - High reliability in lab environments

- Minimal configuration required

- **Exploitation Result:**
  - Successful exploitation
  - Command shell session established on the target system

The successful exploitation demonstrated how outdated public-facing services can lead to complete system compromise.

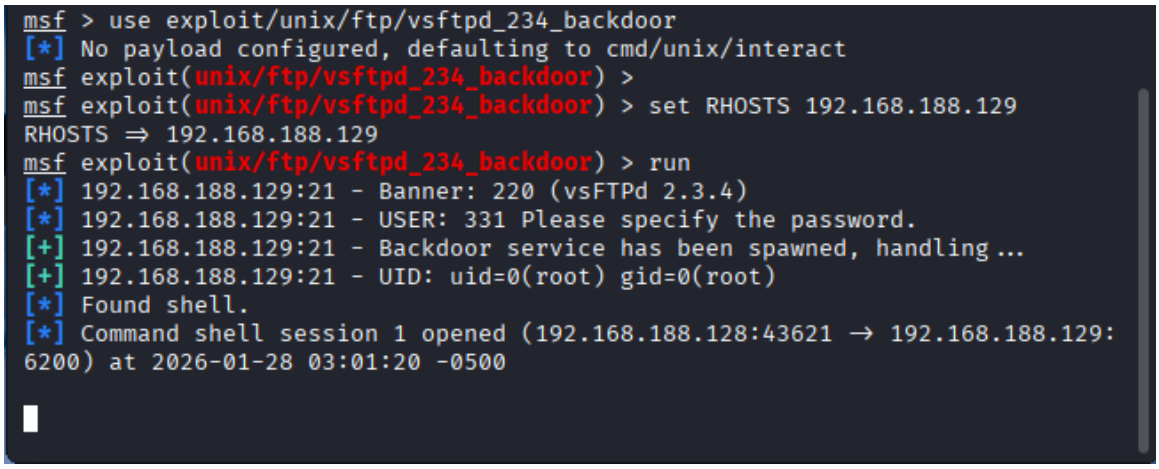

Figure 6: exploit execution and command shell session

4.4 Observations

The exploitation highlights several important security considerations:

- Unpatched services significantly increase attack risk
- Public-facing services are common entry points for attackers
- Vulnerability scanning combined with exploitation can quickly lead to system compromise
- Proper patch management and network segmentation are critical to reducing attack impact

---

5. Encrypted Traffic Analysis

5.1 Traffic Capture Using Wireshark

Encrypted network traffic was captured using Wireshark to observe HTTPS communication patterns without decrypting the payload. The capture was performed in a controlled virtual environment using normal web browsing activity.

- Traffic capture duration: approximately 5 minutes

- Capture interface: active network interface on Kali Linux
- HTTPS websites were accessed to generate encrypted traffic
- TLS-based encrypted packets were observed during the capture

The captured traffic primarily consisted of encrypted HTTPS packets, confirming that application-layer data was protected during transmission.

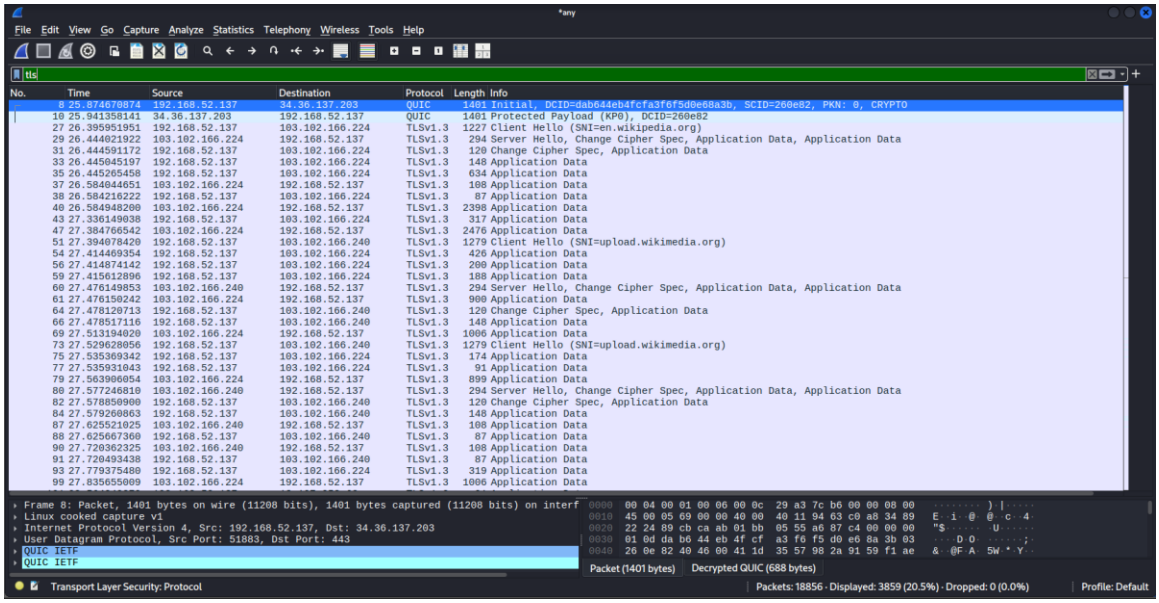

Figure 7: Wireshark capture with TLS/HTTPS packets visible

5.2 Metadata Analysis

Although HTTPS traffic is encrypted, important information can still be analyzed through packet metadata. This metadata provides valuable insights into communication patterns without revealing sensitive content.

- Packet size remains visible even when payloads are encrypted
- Source and destination IP addresses can be identified
- Traffic volume and communication frequency can be analyzed
- Encrypted traffic behavior can be profiled without decryption

This approach allows security analysts to monitor and investigate encrypted traffic while maintaining data confidentiality.

---

6. Python-Based Traffic Analysis

6.1 Tool Selection

Python was used to automate the analysis of encrypted network traffic metadata due to its simplicity and strong ecosystem of networking and visualization libraries.

- **Python**
  Used as the primary programming language for traffic analysis automation.
- **pyshark**
  Utilized to parse Wireshark capture files and extract packet-level metadata.
- **matplotlib**
  Used to visualize packet size distribution through a bar chart.

6.2 Script Explanation

A Python script was developed to analyze encrypted HTTPS traffic by processing packet metadata from the capture file.

- The script loads the Wireshark .pcapng capture file using pyshark
- Each packet is inspected to identify IP-based traffic
- Packet sizes are extracted and stored for analysis
- Packets are counted based on source and destination IP address pairs

This approach enables traffic pattern analysis without decrypting encrypted payloads.

6.3 Python Script

Python Script for Encrypted Traffic Analysis:

```
"""
traffic_analyzer.py

Purpose:
Analyze encrypted HTTPS traffic metadata from a Wireshark capture file.
This script does NOT decrypt traffic. It only analyzes metadata such as
packet size and source/destination IP addresses.

Author: <Your Name>
"""

from typing import Dict, List, Tuple

import pyshark
import matplotlib.pyplot as plt

# Path to the Wireshark capture file
PCAP_FILE: str = "/home/kali/t4/capture/https_traffic.pcapng"

# Load the capture file
capture = pyshark.FileCapture(PCAP_FILE)
```

```
# Data containers
packet_sizes: List[int] = []
ip_pair_counts: Dict[Tuple[str, str], int] = {}

# Iterate through packets
for packet in capture:
    try:
        # We only care about IP packets
        if hasattr(packet, "ip"):
            src_ip: str = packet.ip.src
            dst_ip: str = packet.ip.dst

            # Packet length
            size: int = int(packet.length)
            packet_sizes.append(size)

            # Count packets per source-destination pair
            ip_pair: Tuple[str, str] = (src_ip, dst_ip)
            ip_pair_counts[ip_pair] = ip_pair_counts.get(ip_pair, 0) + 1

    except Exception:
        # Skip packets that cause parsing issues
        continue

# Close the capture to free resources
capture.close()

# Create packet size distribution chart
plt.figure()
plt.hist(packet_sizes, bins=20)
plt.title("Packet Size Distribution (HTTPS Traffic)")
plt.xlabel("Packet Size (bytes)")
plt.ylabel("Number of Packets")

# Save the chart
plt.savefig("packet_size_distribution.png")
plt.close()
```

6.4 Packet Size Distribution

The packet size distribution was visualized to understand the behavior of encrypted HTTPS traffic.

- Small-sized packets were observed at a higher frequency
- Larger packets appeared less frequently, representing encrypted application data
- The distribution reflects typical TLS communication patterns

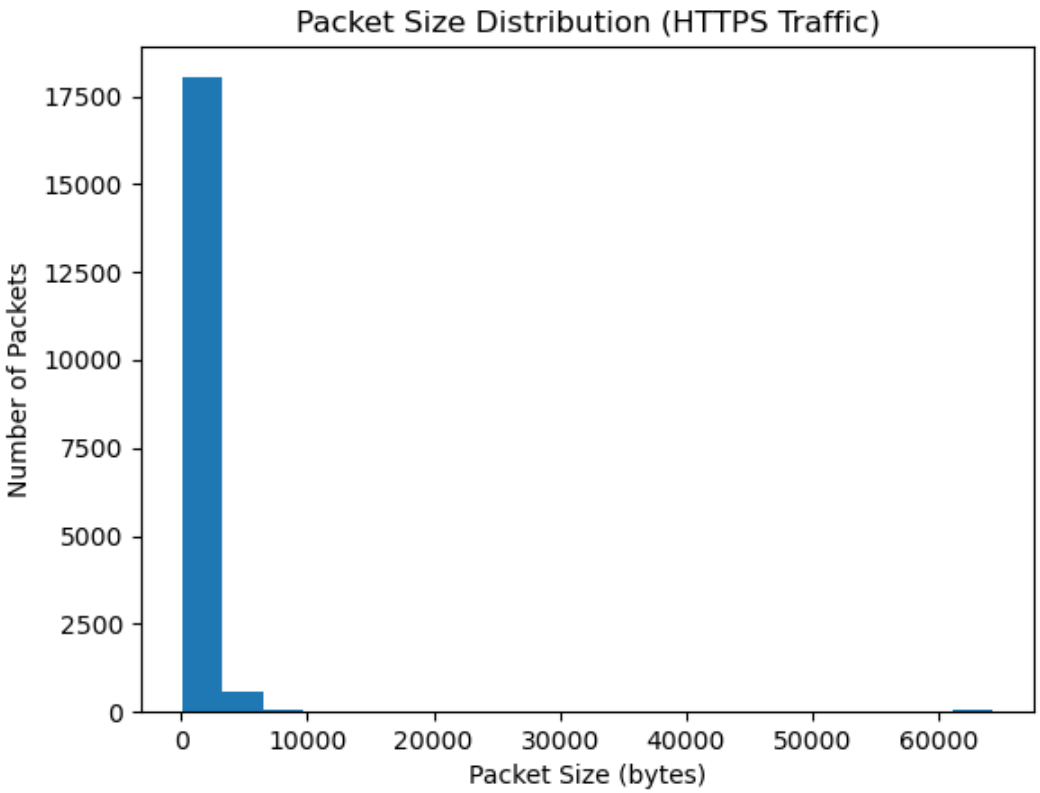

Figure 8: Packet size distribution bar chart here

---

7. Results and Findings

7.1 Vulnerabilities Discovered

The penetration testing phase revealed that the target system exposed multiple services without authentication, increasing the attack surface significantly. A critical vulnerability was identified in the FTP service running vsftpd 2.3.4, which was successfully exploited using Metasploit.

- Vulnerability scanning identified publicly accessible services on the target system
- The vsftpd 2.3.4 service was vulnerable to a known backdoor exploit
- Successful exploitation resulted in a command shell session on the target machine
- The attack demonstrated how unpatched services can lead to full system compromise

This confirms that outdated and exposed services pose a high security risk if not properly managed.

7.2 Encrypted Traffic Behavior Observations

Analysis of the captured HTTPS traffic showed characteristic patterns of encrypted communication. Although the payload content was encrypted, traffic behavior could still be observed through metadata.

- Majority of packets were small in size, indicating TLS handshakes and control messages
- Larger packets occurred less frequently, representing encrypted application data
- Packet payloads remained encrypted throughout the capture
- Source and destination IP addresses remained visible despite encryption

The packet size distribution graph confirms that encrypted traffic still reveals useful behavioral patterns without exposing sensitive content.

7.3 Importance of Segmentation and Monitoring

The results highlight the importance of secure network design and continuous monitoring.

- Successful exploitation of a public-facing service demonstrates the need for DMZ isolation
- If the vulnerable service were placed within the internal network, the impact would be significantly higher
- Network segmentation limits lateral movement after initial compromise
- Monitoring encrypted traffic metadata provides visibility even when payload inspection is not possible

Together, these findings reinforce the importance of combining segmentation, patch management, and traffic monitoring for effective network security.

---

8. Key Learnings

- **Secure Network Design:**
  This task reinforced the importance of network segmentation using firewalls, DMZs, and VLANs to reduce attack impact. Proper isolation ensures that even if a public-facing service is compromised, internal systems remain protected.

- **Penetration Testing:**
  Performing vulnerability scanning and exploitation demonstrated how quickly outdated and exposed services can be compromised. The exercise highlighted the critical role of regular patching and controlled penetration testing in identifying real-world security weaknesses.
- **Challenges and Resolution:**
  Challenges were faced in configuring the lab environment and capturing encrypted traffic correctly. These were overcome by adjusting virtual network configurations and analyzing traffic metadata instead of attempting payload decryption.

---

9. Sources and References

- **Metasploit Framework Documentation**
  https://docs.metasploit.com
- **Wireshark User Guide and Documentation**
  https://www.wireshark.org/docs
- **pyshark Python Library Documentation**
  https://github.com/KimiNewt/pyshark
- **OWASP Security Best Practices**
  https://owasp.org
- **Nmap Network Scanning Documentation**
  https://nmap.org/book/man.html

---

10. Conclusion

This task successfully demonstrated the practical application of core cybersecurity concepts, including secure network design, controlled penetration testing, and encrypted traffic analysis. By designing a segmented network topology, conducting vulnerability scanning and exploitation using Metasploit, and analyzing HTTPS traffic metadata with Wireshark and Python, the task highlighted how security weaknesses can be identified and mitigated in a structured manner.

The outcomes reflect real-world cybersecurity operations where attackers exploit exposed services, defenders rely on segmentation to limit impact, and analysts monitor encrypted traffic through metadata rather than payload inspection. Overall, the task reinforced the importance of combining preventive controls, offensive testing, and continuous monitoring to maintain a resilient and secure network environment.